JavaScript_Day – 3_Week2_Hands_On -- Harshitha Kamatam

Problem 1

Problem Statement: Interactive Feedback Button

Scenario:

A startup company wants a simple interactive webpage where users can click a button to submit feedback and instantly see a confirmation message without refreshing the page.

Requirements

- Create a webpage with a “Submit Feedback” button.
- When clicked, display a confirmation message dynamically.
- Use JavaScript event listeners to handle the click event.

Technical Constraints

- Use plain HTML, CSS, and JavaScript.
- Event handling must use addEventListener().
- No external frameworks allowed.

Expectations:

- Proper DOM manipulation.
- Clean and readable code.

Learning Outcome

- Understand browser event handling.
- Learn DOM manipulation basics.

Source Code:

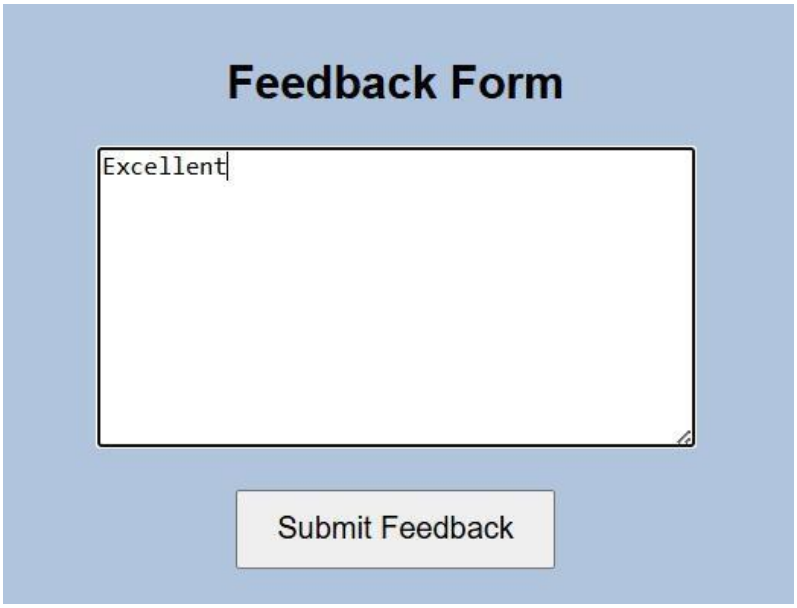
File Name: Index.html

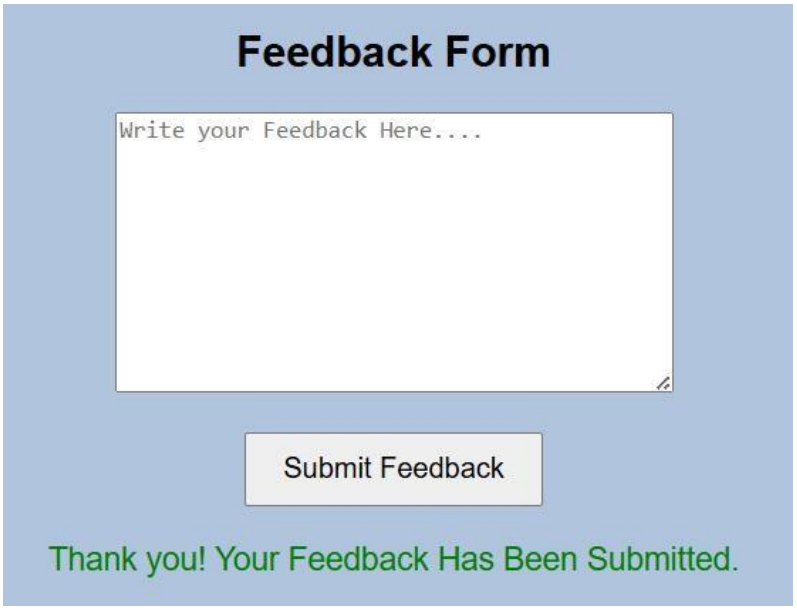
p1 > index.html > ...

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Interactive Feedback</title>
<style>
body {
  font-family: Arial, Helvetica, sans-serif;
  text-align: center;
  margin-top: 100px;
  background-color: lightsteelblue;
}
button {
  padding: 10px 20px;
  font-size: 16px;
}
#message {
  margin-top: 20px;
  font-size: 18px;
}
</style>
</head>
<body>
<h2>Feedback Form</h2>
<textarea rows="10" cols="40" placeholder="Write your Feedback Here..."></textarea><br><br>
<button id="feedbackbtn">Submit Feedback</button>
<p id="message"></p>
<script>
const button = document.getElementById("feedbackbtn");
const message = document.getElementById("message");
const textarea = document.querySelector("textarea");
button.addEventListener("click", function () {
  if (textarea.value.trim() === "") {
    message.textContent = "Please Enter Your Feedback Before Submitting.";
    message.style.color = "red";
  } else {
    message.textContent = "Thank you! Your Feedback Has Been Submitted.";
    message.style.color = "green";
    textarea.value = "";
  }
});
</script>
</body>
</html>
```

Output:

Output File: index.html





Explanation:

This code creates a simple **Feedback Form** where the user can type feedback and click **Submit Feedback**. JavaScript checks if the textarea is empty before submitting. If no feedback is entered, it shows a **red error message** asking the user to enter feedback. If feedback is provided, it displays a **green success message** and clears the textbox.

Problem 2

Problem Statement: Theme Toggle Application (Level-1)

Scenario:

A small blog website wants to allow users to switch between Light Mode and Dark Mode interactively.

Requirements

- Create a toggle button to switch themes.
- Change background and text color dynamically.
- Store theme preference in localStorage.

Technical Constraints

- Use Vanilla JavaScript.
- Use addEventListener() for events.
- No CSS frameworks.

Expectations:

- Smooth theme switching.
- Proper usage of localStorage.
- Clean and modular JavaScript functions.

Learning Outcome

- Handling user interaction events.
- Managing browser storage.

Source Code:

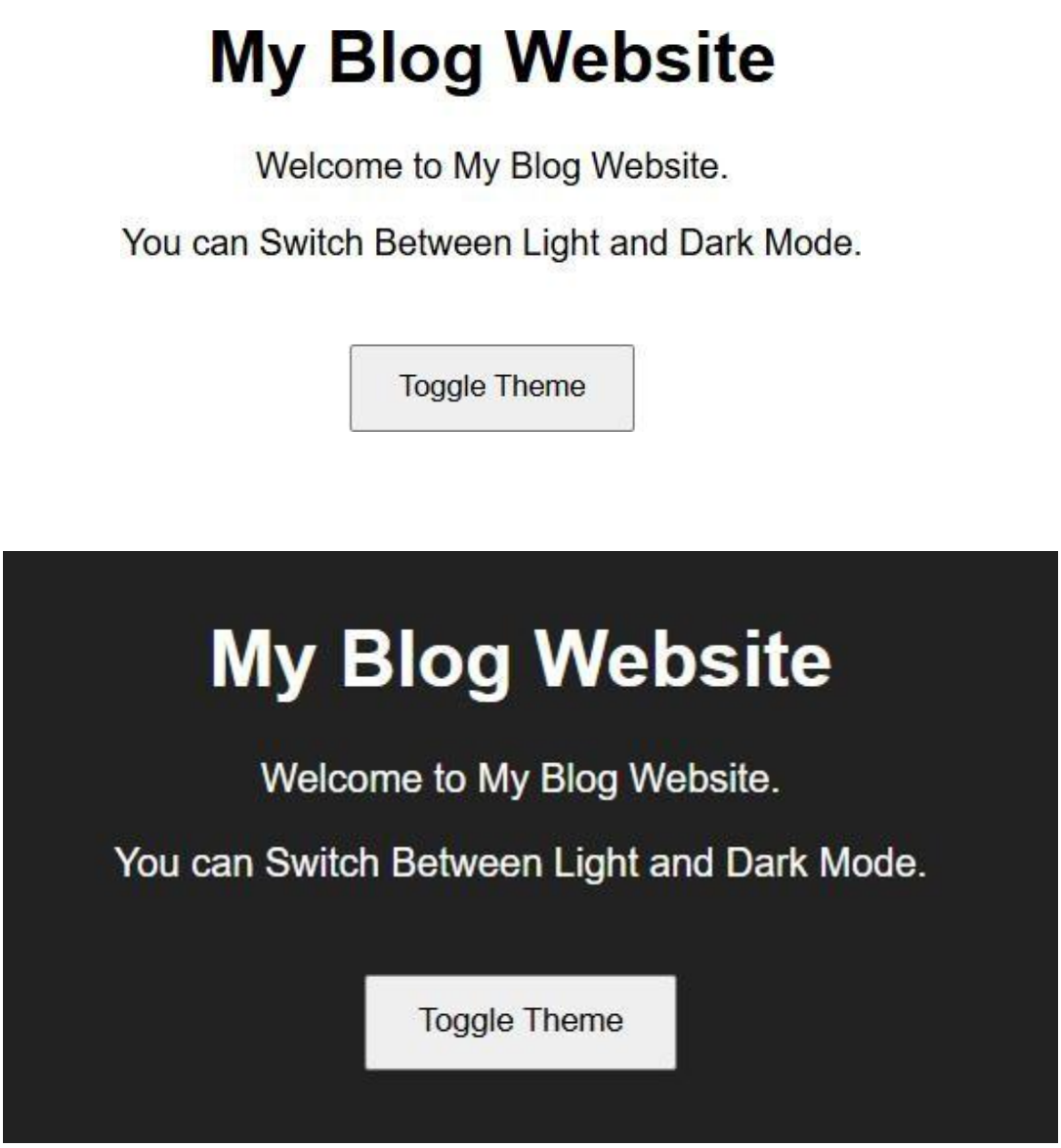
File Name: index.html

p2 > index.html > ...

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Theme Toggle Application</title>
<style>
body {
  font-family: Arial;
  background-color: white;
  color: black;
  text-align: center;
  padding-top: 100px;
  transition: 0.3s;
}
.dark-mode {
  background-color: #222;
  color: white;
}
button {
  padding: 10px 20px;
  margin-top: 20px;
  cursor: pointer;
}
</style>
</head>
<body>
<div>
<h1>My Blog Website</h1>
<p>Welcome to My Blog Website.</p>
<p>You can Switch Between Light and Dark Mode.</p>
</div>
<button id="themeToggle">Toggle Theme</button>
<script>
const toggleBtn = document.getElementById("themeToggle");
const body = document.body;
// Load saved theme
const savedTheme = localStorage.getItem("theme");
if (savedTheme === "dark") {
  body.classList.add("dark-mode");
}
// Toggle theme
toggleBtn.addEventListener("click", () => {
  body.classList.toggle("dark-mode");
  if (body.classList.contains("dark-mode")) {
    localStorage.setItem("theme", "dark");
  } else {
    localStorage.setItem("theme", "light");
  }
});
</script>
</body>
</html>
```

Output:

Output File: index.html



Explanation:

This code creates a simple **Theme Toggle webpage** where the user can switch between **Light mode and Dark mode** by clicking the **Toggle Theme** button. JavaScript adds or removes the **dark-mode class** to change the background and text color, and it also saves the selected theme in **localStorage** so the same theme stays when the page is refreshed.

Problem 3

Problem Statement: Dynamic To-Do List (Level-2)

Scenario:

A productivity application requires an interactive To-Do list where users can add, delete, and mark tasks as complete without refreshing the page.

⚡ Requirements

- Add new tasks dynamically.
- Delete tasks on button click.
- Mark tasks as completed.
- Use event delegation for better performance.

⚔️ Technical Constraints

- Must use DOM manipulation.
- Event delegation required.
- Code should follow modular structure.

Expectations:

- Fully functional interactive UI.
- Organized and reusable functions.

🎯 Learning Outcome

- Advanced DOM event handling.
- Event delegation concept.
- Understanding role of libraries in speeding up development.

Source Code:

File Name: index.html

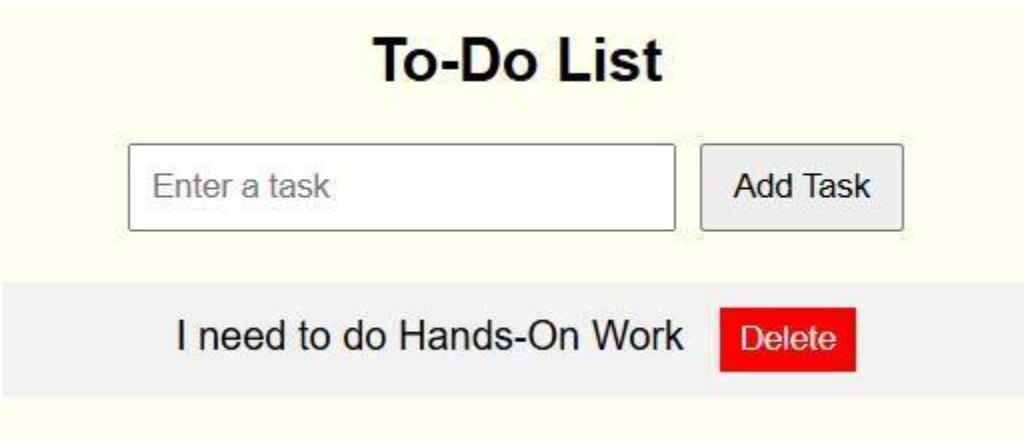
p4 > index.html > html

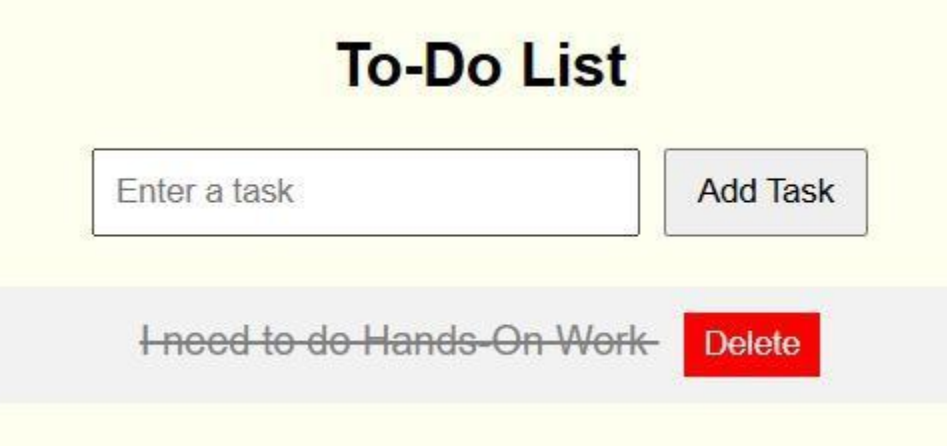
```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Mini Product Search App</title>
<style>
body {
  font-family: Arial;
  text-align: center;
  margin-top: 50px;
  background-color: lightcyan;
}
input {
  padding: 8px;
  width: 250px;
  margin-bottom: 20px;
}
ul {
  list-style: none;
  padding: 0;
}
li {
  background: #f2f2f2;
  margin: 5px;
  padding: 10px;
}
li:hover {
  background: #ddd;
  cursor: pointer;
}
</style>
</head>
<body>
<h2>Product Search</h2>
<input type="text" id="searchBox" placeholder="Search products...">
<ul id="productList"></ul>
<p id="noResult" style="display:none;">No Results Found</p>
<script>
const searchBox = document.getElementById("searchBox");
const productList = document.getElementById("productList");
const noResult = document.getElementById("noResult");
const products = [
  "Laptop",
  "Mobile Phone",
  "Headphones",
  "Keyboard",
  "Mouse",
  "Smart Watch",
  "Tablet"
];
const displayProducts = (items) => {
  productList.innerHTML = "";
  if (items.length === 0) {
```

```
    if (items.length === 0) {
      noResult.style.display = "block";
      return;
    }
    noResult.style.display = "none";
    items.forEach(product => {
      const li = document.createElement("li");
      li.textContent = product;
      productList.appendChild(li);
    });
  };
  const filterProducts = () => {
    const searchText = searchBox.value.toLowerCase();
    const filtered = products.filter(product => product.toLowerCase().includes(searchText));
    displayProducts(filtered);
  };
  searchBox.addEventListener("input", filterProducts);
  productList.addEventListener("click", (event) => {
    if (event.target.tagName === "LI") {
      alert(`You selected: ${event.target.textContent}`);
    }
  });
  displayProducts(products);
  </script>
  </body>
  </html>
```

Output:

Output File: index.html





Explanation:

This code creates a simple **To-Do List web app** where users can type a task and click **Add Task** to display it in the list. When a task is added, it appears with a **Delete button**, and clicking the task marks it as **completed (line-through)**. Users can also **remove tasks** by clicking the delete button.

Problem 4

Problem Statement: Mini Product Search App (Level-2)

Scenario:

An e-commerce company needs a mini product search application that filters products dynamically as users type in the search box.

Requirements

- Display product list from a predefined array.
- Filter products dynamically using input event.
- Display “No Results Found” when applicable.

Technical Constraints

- Must use DOM manipulation.
- Event delegation required.
- Code should follow modular structure.

Expectations:

- Real-time filtering.
- Proper state management.

Learning Outcome

- State management basics.

Source Code:

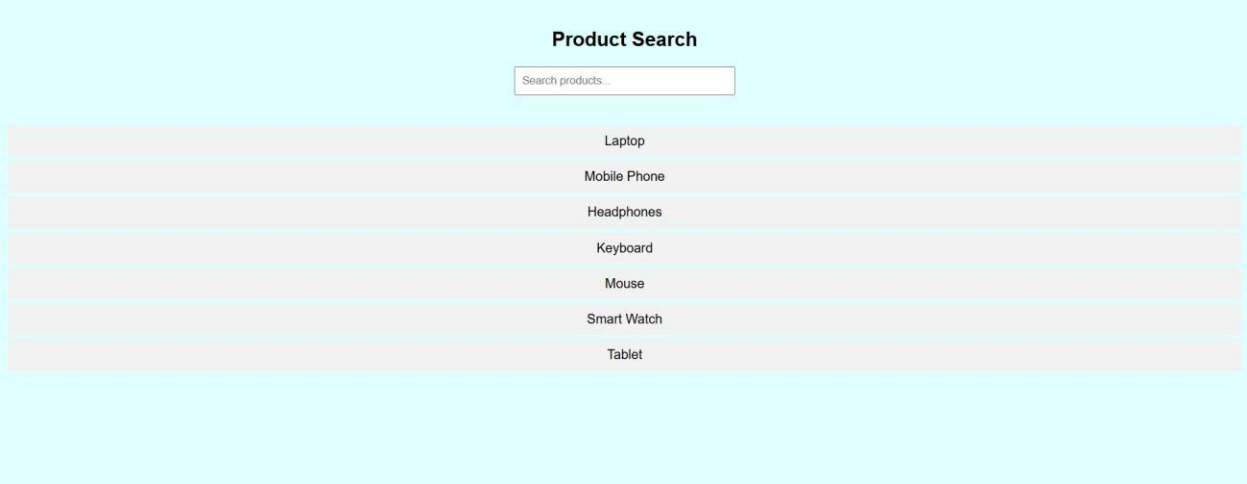
File Name: index.html

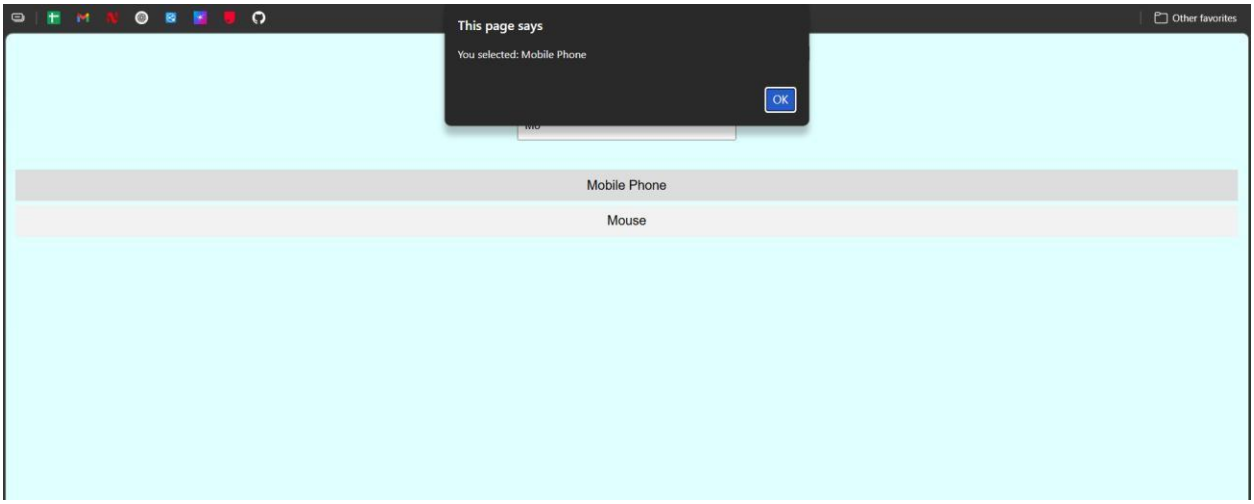
```
p4 > <> index.html > ...
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini Product Search App</title>
<style>
body {
  font-family: Arial;
  text-align: center;
  margin-top: 50px;
  background-color: lightcyan;
}
input {
  padding: 8px;
  width: 250px;
  margin-bottom: 20px;
}
ul {
  list-style: none;
  padding: 0;
}
li {
  background: #f2f2f2;
  margin: 5px;
  padding: 10px;
}
li:hover {
  background: #ddd;
  cursor: pointer;
}
</style>
</head>
<body>
  <h2>Product Search</h2>
  <input type="text" id="searchBox" placeholder="Search products...">
  <ul id="productList"></ul>
  <p id="noResult" style="display:none;">No Results Found</p>
<script>
const searchBox = document.getElementById("searchBox");
const productList = document.getElementById("productList");
const noResult = document.getElementById("noResult");
const products = [
  "Laptop",
  "Mobile Phone",
  "Headphones",
  "Keyboard",
  "Mouse",
  "Smart Watch",
  "Tablet"
];
const displayProducts = (items) => {
  productList.innerHTML = "";
  if (items.length === 0) {
```

```
  if (items.length === 0) {
    noResult.style.display = "block";
    return;
  }
  noResult.style.display = "none";
  items.forEach(product => {
    const li = document.createElement("li");
    li.textContent = product;
    productList.appendChild(li);
  });
};
const filterProducts = () => {
  const searchText = searchBox.value.toLowerCase();
  const filtered = products.filter(product => product.toLowerCase().includes(searchText));
  displayProducts(filtered);
};
searchBox.addEventListener("input", filterProducts);
productList.addEventListener("click", (event) => {
  if (event.target.tagName === "LI") {
    alert(`You selected: ${event.target.textContent}`);
  }
});
displayProducts(products);
</script>
</body>
</html>
```

Output:

Output File: index.





Explanation:

This code creates a simple **Product Search app** where a list of products is shown on the page. When the user types something in the search box, JavaScript **filters the products in real time** and only matching items are displayed. If a user clicks on any product, an **alert message shows the selected product name**, and if nothing matches the search, it displays **“No Results Found.”**